Advanced IoT Systems & Design: Laboratory Manual

Introduction

This manual contains ten 3-hour labs, one per lecture from L02 to L10, that build the IoT system used in the class project. The Arduino Nano 33 BLE Sense from Advanced Microprocessors is the intelligent sensor node; an ESP32 is the gateway to Wi-Fi and the cloud.

Kit per group

Item	Qty	Used in
Arduino Nano 33 BLE Sense Rev2 (with headers) and micro-USB cable	1	Labs 1 to 10
ESP32 development board (e.g. ESP32-DevKitC) and USB cable	1	Labs 3, 4, 5, 9, 10
28BYJ-48 stepper motor with ULN2003 driver board	1	Lab 8
External 5 V, 2 A power supply (or USB power bank with breakout)	1	Lab 8
Capacitive soil moisture sensor v1.2	1	Lab 2
INA219 current sensor module	1	Lab 3
Breadboard, jumper wires, push button, LEDs, resistors	1 set	All labs

On-board sensors of the Nano 33 BLE Sense Rev2. BMI270 and BMM150 (9-axis IMU), HS3003 (temperature and humidity), LPS22HB (pressure), APDS-9960 (light, colour, proximity and gesture) and MP34DT06JTR (PDM microphone). Rev1 boards use LSM9DS1, HTS221 and MP34DT05 instead, with different Arduino libraries.

Software (all free). Arduino IDE 2 with the *Arduino Mbed OS Nano Boards* and *esp32* board packages; Arduino CLI; Edge Impulse account and Edge Impulse CLI (Node.js required); Python 3 with Jupyter Notebook (or Google Colab), TensorFlow, pandas, matplotlib and paho-mqtt; Mosquitto; Docker Desktop; Git and GitHub; nRF Connect mobile app.

Safety and care. The Nano 33 BLE Sense uses 3.3 V I/O and is **not 5 V tolerant**: never connect a 5 V signal to its pins. Power the stepper motor from the external 5 V supply, never from the Nano, and always share a common ground. Handle Li-ion cells only in protected holders. Disconnect power before rewiring.

Lab reports. One report per student within one week of each lab: aim, setup with wiring diagram or photo, method, results with units, discussion, and answers to the post-lab questions, with a link to your Git repository. Reports are marked out of 20 using the rubric in the Assessment Plan.

Lab 0: Self-Study Preparation (before Week 2)

Aim. Give students who did not take Advanced Microprocessors the Arduino, Python and Git basics the labs assume.

Tasks

1. Install Arduino IDE 2, Python 3 with Jupyter, and Git. Create a GitHub account and a private repository named `aiot-labs-<index number>`; add the lecturer as a collaborator.
2. Work through chapters 1 to 4 of Simon Monk, *Programming Arduino* (sketches, variables, functions, digital and analog I/O).
3. Revise Python lists, functions and NumPy arrays in a Jupyter notebook.
4. Create a free Edge Impulse account at edgeimpulse.com.

Deliverable. A screenshot of the Blink example compiled for the Nano 33 BLE and the link to your repository, submitted before the Week 2 lab. Lab 0 is not graded.

Lab 1: Toolchain Setup and On-Board Sensors (Week 2, L02)

Objectives. Set up the Arduino and Edge Impulse toolchains for both boards; read every on-board sensor of the Nano 33 BLE Sense; use interrupts and timing without blocking.

Equipment. Nano 33 BLE Sense Rev2, ESP32 board, push button, LED, 220 Ω resistor, laptop.

Background. The Nano 33 BLE Sense is built on the Nordic nRF52840 (Arm Cortex-M4F at 64 MHz, 1 MB flash, 256 KB RAM) and runs Arm Mbed OS underneath the Arduino core. Its I/O pins are 3.3 V only.

Procedure

1. Install the *Arduino Mbed OS Nano Boards* package and flash Blink to the Nano. Double-press RESET to enter the bootloader if upload fails.
2. Install the *esp32* board package and flash Blink to the ESP32. Record both build sizes (flash and RAM) from the compiler output.
3. Install the sensor libraries (Arduino_BMI270_BMM150, Arduino_HS300x, Arduino_LPS22HB, Arduino_APDS9960) and print accelerometer, temperature, humidity, pressure and light readings once per second on the Serial Monitor.
4. Record 2 seconds of audio with the PDM library and print the RMS level; clap to see it change.
5. Wire a push button to D2 and toggle an LED on D3 using `attachInterrupt()` with software debouncing; replace every `delay()` in your sketch with `millis()`-based timing.
6. Install Node.js and the Edge Impulse CLI; run `edge-impulse-daemon` to confirm your laptop can connect a board to your project (full data collection comes in Lab 7).
7. Commit all sketches to Git.

Post-lab questions

1. Compare the nRF52840 and ESP32 in core, clock, memory and radios. Which is the better sensor node and which the better gateway, and why?
2. Why does blocking code with `delay()` cause problems in a device that must sample sensors and talk over a radio at the same time?
3. What would happen if you connected a 5 V sensor output directly to a Nano 33 BLE Sense pin? How would you interface it safely?

Lab 2: Sensor Data Logging and Calibration (Week 3, L03a)

Objectives. Stream sensor data from the Nano to a Jupyter notebook, analyse noise, and calibrate a low-cost soil moisture sensor against a reference measurement.

Equipment. Nano 33 BLE Sense, capacitive soil moisture sensor, two soil samples in cups, water, kitchen scale, oven or sun-drying space, laptop with Jupyter.

Procedure

1. Write a sketch that prints the IMU, temperature, humidity and pressure readings as comma-separated values at 10 Hz.
2. In Jupyter, read the serial stream with `pyserial`, save 2 minutes to CSV with pandas, and plot each channel. Compute the mean and standard deviation of each channel with the board at rest.
3. Compare your temperature and humidity with two other groups' boards for 5 minutes and report the spread. Explain why the on-board temperature reads higher than room temperature (self-heating).
4. Power the soil sensor from the Nano's 3.3 V pin and connect its output to A0. Set `analogReadResolution(12)`. Record the raw value in air and in water.
5. Prepare soil at four moisture levels. For each, take 20 readings, then find the true water content by weighing the sample before and after drying.
6. Fit a calibration curve in Jupyter and implement it in the sketch so the Nano outputs moisture in percent.

Post-lab questions

1. Report your calibration equation and R^2. How would you check it on a different soil type?
2. How would you correct the self-heating error of the on-board temperature sensor in a deployed node?
3. Why is it important to log raw values as well as calibrated values in an IoT system?

Lab 3: Power Measurement and Battery Life (Week 4, L03b)

Objectives. Measure the current profile of the Nano node and the ESP32 gateway in each operating mode, apply duty cycling, and estimate battery life from measurements.

Equipment. Nano 33 BLE Sense, ESP32, INA219 module, 5 V USB power bank with breakout, multimeter, second board or laptop for logging.

Background. Average current = sum of (current × time) for each state, divided by the period. Development boards draw extra current through their regulator, USB circuitry and power LED, so measured values exceed the chip datasheet.

Procedure

1. Place the INA219 in series between the 5 V supply and the Nano's VIN pin. Log current at the highest rate available.
2. Measure the Nano while (a) idle in `loop()`, (b) reading all sensors, (c) recording audio with the PDM microphone, (d) advertising over BLE and (e) idle with sensors powered down using each library's `end()` call.
3. Write a duty-cycled sketch: wake every 60 s, read temperature and humidity, advertise for 2 s, then idle. Capture one full cycle and calculate the charge per cycle.
4. Repeat on the ESP32: measure Wi-Fi connect, transmit and deep sleep (`esp_deep_sleep_start()` with a timer wake-up).
5. Estimate battery life for a 2600 mAh pack at 1-minute, 10-minute and 60-minute intervals for both boards.

Post-lab questions

1. Which state dominates energy use at each interval for each board?
2. Why is the battery-powered node in our architecture the Nano (BLE) and the mains or solar-powered device the ESP32 gateway?
3. Would a 6 V, 1 W solar panel keep the 10-minute Nano node running indefinitely in Accra? State your assumptions about peak sun hours.

Lab 4: BLE Sensor Node and ESP32 Gateway (Week 5, L04a)

Objectives. Turn the Nano into a BLE sensor node, receive its data on the ESP32 acting as a BLE central, and measure link quality.

Equipment. Nano 33 BLE Sense, ESP32, smartphone with nRF Connect, power bank. Optional: two SX1276 LoRa modules.

Procedure

1. With the ArduinoBLE library, make the Nano a peripheral that exposes an Environmental Sensing service (UUID 0x181A) with temperature (0x2A6E) and humidity (0x2A6F) characteristics, updated every 5 s with notifications.
2. Connect with nRF Connect, subscribe to notifications and check the values against the Serial Monitor.
3. Program the ESP32 as a BLE central (NimBLE-Arduino library) that scans for the Nano by name, connects, subscribes and prints each notification.
4. Add a writable "sampling interval" characteristic on the Nano so the gateway can change how often it reports.
5. Record RSSI and the fraction of notifications received at 1 m, 5 m, 10 m and through one wall. Change the advertising interval from 100 ms to 1000 ms and note the effect on discovery time and Nano current (Lab 3 setup).

6. Optional extension: link two ESP32 boards with SX1276 modules at 868 MHz and compare range at SF7 and SF12.

Post-lab questions

1. Draw the GATT structure of your node (services, characteristics, properties).
2. How many Nano nodes could one ESP32 gateway serve, and what limits that number?
3. For a farm where nodes are 2 km from the gateway, which link would you choose instead of BLE, and what changes in the architecture?

Lab 5: Gateway to MQTT with FreeRTOS (Week 6, L04b)

Objectives. Structure the gateway firmware as FreeRTOS tasks, forward Nano data to an MQTT broker, and send commands back down to the node.

Equipment. Nano node and ESP32 gateway from Lab 4, laptop running Mosquitto and Jupyter, all on the same Wi-Fi network.

Procedure

1. Run Mosquitto on the laptop with a configuration file containing `listener 1883` and `allow_anonymous true` (Mosquitto 2.x accepts only local connections otherwise). Anonymous access is for this lab only; Lab 9 removes it.
2. On the ESP32, create two FreeRTOS tasks: `bleTask` receives notifications from the Nano and places readings in a queue; `mqttTask` connects to Wi-Fi and publishes each reading as JSON to `gctu/aiot/<group>/env` (PubSubClient library).
3. Print stack high-water marks with `uxTaskGetStackHighWaterMark()` and size each task's stack.
4. In Jupyter, subscribe with paho-mqtt, log messages to CSV and plot them live.
5. Subscribe the gateway to `gctu/aiot/<group>/cmd`; when a new sampling interval arrives, write it to the Nano's characteristic from Lab 4. Use a retained message for the current setting.
6. Configure a Last Will and Testament for the gateway, then unplug it and observe the message.
7. Publish 50 messages at QoS 0 and 50 at QoS 1 from a Python paho-mqtt client (PubSubClient on the gateway only publishes at QoS 0), capture them in Wireshark and compare bytes on the wire.

Post-lab questions

1. Draw your end-to-end data path from sensor to notebook, naming every protocol.
2. Why separate BLE handling and MQTT publishing into different tasks? What happens to readings while Wi-Fi is reconnecting?
3. Compare MQTT with CoAP for this gateway. When would CoAP be the better choice?

Lab 6: Machine Learning in Jupyter: DNN, CNN and Quantization (Weeks 7 and 8, L05 and L06)

Objectives. Put the ideas of the TinyML and deep learning lectures into practice: loss and gradient descent, DNN and CNN classifiers, performance metrics, and shrinking a model for a microcontroller.

Equipment. Laptop with Jupyter and TensorFlow, or Google Colab. No board needed until the final step.

Part 1, Week 7 (The Machine Learning Paradigm)

1. Fit a straight line to a small dataset by hand-coding mean squared error and gradient descent in NumPy. Plot the loss against iterations for three learning rates.
2. Repeat with a one-neuron Keras model and compare the learned weight and bias.

Part 2, Week 8 (Classification, DNN and ML Metrics, Convolutions)

1. Train a DNN on MNIST. Split the data into training, validation and test sets and plot training and validation accuracy to detect overfitting.
2. Produce the confusion matrix on the test set and compute accuracy, precision, recall and F1 for two digits of your choice.
3. Train a small CNN on CIFAR-10 and compare it with a DNN of similar size, as in the Convolutions lecture.
4. Convert the CNN to TensorFlow Lite as float32, dynamic-range and full int8 models (with a representative dataset). Tabulate file size and test accuracy for each.
5. Convert the int8 model to a C array with `xxd -i` and compile it in an empty Nano 33 BLE sketch to check that it fits in flash and RAM.

Post-lab questions

1. Why did the CNN outperform the DNN on CIFAR-10 but not by much on MNIST?
2. Explain the accuracy and size change from float32 to int8. Is the trade-off worth it on a Cortex-M4?
3. For a crop disease detector, which matters more, precision or recall? Justify your answer.

Lab 7: Edge Impulse Motion Classifier on the Nano (Week 9, L07)

Objectives. Follow the full TinyML workflow in Edge Impulse (collect, preprocess, design, train, evaluate, convert, deploy), run inference on the Nano 33 BLE Sense, and send results through the gateway.

Equipment. Nano 33 BLE Sense, ESP32 gateway from Lab 5, laptop with Edge Impulse CLI. Optional: a small fan or water pump to classify.

Scenario. Classify the vibration of a small fan or pump (off, normal, blocked or imbalanced), a stand-in for monitoring irrigation pumps. If no motor is available, classify hand gestures (idle, circle, shake, tap).

Procedure

1. Flash the Edge Impulse firmware for the Nano 33 BLE Sense, run `edge-impulse-daemon`, and connect the board to a new project.
2. Collect at least 3 minutes of accelerometer data per class at 100 Hz from the Data acquisition page, with an 80/20 train and test split.
3. Create an impulse with 2-second windows, a Spectral Analysis block and a Classification block. Inspect the feature explorer: are the classes separable?
4. Train the neural network, record accuracy and the confusion matrix, then run Model testing on the test set.
5. Add an Anomaly Detection (K-means) block and test it with a motion the model never saw. Note the anomaly score.

6. Compare the float32 and int8 (quantized) on-device performance estimates for RAM, flash and latency.
7. Deploy the int8 model as an Arduino library, run the accelerometer example on the Nano and measure latency with `micros()`.
8. Send the predicted class and anomaly score over BLE to the gateway, which publishes them over MQTT. Compare bytes per minute with streaming raw accelerometer data.

Post-lab questions

1. Report test accuracy, on-device accuracy, latency, RAM and flash for float32 and int8.
2. Using your Lab 3 measurements, how much radio energy does sending one result every 2 s save compared with streaming raw data?
3. How does the anomaly block complement the classifier in a real monitoring system?

Lab 8: Voice-Controlled IoT Actuator (Week 10, L08)

Objectives. Extend the L08 hands-on project: train a keyword-spotting model on the Speech Commands dataset, deploy it to the Nano 33 BLE Sense to drive a stepper motor, and connect the actuator to the IoT system so it can also be monitored and commanded remotely.

Equipment. Nano 33 BLE Sense, 28BYJ-48 stepper motor, ULN2003 driver board, external 5 V supply, ESP32 gateway from Lab 5, laptop with Edge Impulse CLI and Arduino CLI.

Part A: Wire and test the actuator

1. Connect the external 5 V supply to the ULN2003 VCC and GND; connect IN1, IN2, IN3 and IN4 to Nano pins D8, D9, D10 and D11; plug in the motor; join the grounds.
2. Using the built-in Stepper library, create `Stepper myStepper(2048, 8, 10, 9, 11);` (step sequence IN1-IN3-IN2-IN4) and turn the motor one revolution each way.

Part B: Train and deploy the keyword model

1. From Speech Commands v0.02, upload about 1,500 one-second clips each of "forward" and "backward" plus a noise/silence class built from `_background_noise_` to a new Edge Impulse project.
2. Create an impulse with an MFCC processing block and a 1D convolutional classifier. Compare the MFCC features of the three classes.
3. Train, run Model testing, and try Live classification from the board's microphone.
4. Deploy as a quantized (int8) Arduino library and adapt the continuous microphone example: when "forward" or "backward" is detected above a 0.8 confidence threshold, turn the motor 512 steps (90°) in that direction.

Part C: Connect it to the IoT system

1. Notify each recognised command, its confidence and the motor position over BLE to the gateway, which publishes them to `gctu/aiot/<group>/actuator`.
2. Let the gateway forward a `forward` or `backward` command from MQTT to the Nano, so the motor can be driven remotely as well as by voice.
3. Measure the time from the end of a spoken word to motor movement over 20 trials.

Extension. Record 50 samples each of two keywords in a Ghanaian language (for example Twi or Ga) from several speakers, add them as new classes, and report accuracy on speakers not in the training set.

Post-lab questions

1. Report accuracy, false activations per 10 minutes of background talk, and the voice-to-motion latency.
2. Why does the 28BYJ-48 need the ULN2003 and an external supply? What would happen if the Nano powered it directly?
3. Speech Commands speakers are mostly not Ghanaian. How would this affect a deployed system, and how would you fix it?

Lab 9: Securing the System with TLS and ASCON (Week 11, L09)

Objectives. Threat-model the system built so far, secure the gateway-to-broker link with TLS, and add end-to-end ASCON encryption from the Nano to the backend.

Equipment. Nano node and ESP32 gateway, laptop with Mosquitto, OpenSSL, Python and Wireshark.

Part A: Threat model and TLS

1. Draw a data-flow diagram of your system (Nano, BLE, ESP32, Wi-Fi, broker, notebook) and list at least one threat per STRIDE category.
2. Show in Wireshark that your Lab 5 MQTT traffic is readable in plain text.
3. With OpenSSL, create a lab CA, a broker certificate and a client certificate for the gateway. Configure Mosquitto on port 8883 with TLS only, `allow_anonymous false`, client certificates required, and an ACL limiting each gateway to its own topics.
4. Connect the ESP32 with `WiFiClientSecure` on port 8883 and confirm in Wireshark that payloads are no longer readable. Record the extra connection time and heap use.

Part B: ASCON on the Nano

1. Add the Ascon-AEAD128 reference C implementation (github.com/ascon/ascon-c) to a Nano sketch and verify it against two official test vectors.
2. Benchmark Ascon-AEAD128 against software AES-128-GCM (Arduino Crypto library) for 16, 64 and 128-byte messages: cycles per byte (using the Cortex-M4 DWT cycle counter), flash and RAM.
3. Encrypt each BLE payload on the Nano with Ascon-AEAD128, using the node ID as associated data and a unique nonce per message. Forward the ciphertext unchanged through the gateway and decrypt it in Python (the `ascon` package) on the backend.

Post-lab questions

1. Which STRIDE threats does TLS mitigate, and which does end-to-end ASCON add protection against?
2. Present your benchmark table. Why is ASCON attractive for a Cortex-M4 node?
3. Propose a nonce scheme that never repeats, even after the Nano reboots.

Lab 10: Dashboard, OTA Updates and Digital Twin (Week 12, L10)

Objectives. Build a self-hosted pipeline from the MQTT broker to a dashboard, update gateway firmware over the air, and keep a digital twin of each node.

Equipment. Complete Nano and ESP32 system from Lab 9, laptop with Docker Desktop.

Procedure

1. With Docker Compose, start Mosquitto, Node-RED, InfluxDB 2 and Grafana on one network.
2. In Node-RED, subscribe to your topics, decrypt the ASCON payloads (or use plain JSON if Lab 9 Part B is not complete), validate them and write them to InfluxDB.
3. In Grafana, show environmental readings, the Lab 7 classifier output and anomaly score, the Lab 8 actuator position, and a "last seen" panel per device.
4. Add alerts for an anomaly score above a threshold and for a node silent for 5 minutes.
5. Implement HTTP OTA on the ESP32 with two OTA partitions and rollback if the new image fails its self-test. Deploy a new version over the air.
6. Build a digital twin flow in Node-RED that stores each node's latest readings and settings (sampling interval, firmware version, battery estimate) and pushes setting changes down as retained command messages.
7. Switch off Wi-Fi for 30 minutes; buffer readings on the gateway and upload them with their original timestamps when the link returns.

Post-lab questions

1. Draw your end-to-end architecture, labelling every protocol and port.
2. The Nano itself is not updated over the air in this lab. How would you update models and firmware on hundreds of deployed Nano nodes?
3. Compare this self-hosted stack with a managed cloud IoT platform in cost, data sovereignty and reliability for a Ghanaian deployment.